Purchase Tracking: What to Implement (Thermer Android)

Nov 5, 2026 · For the Thermer developer

Goal. Every real Play Store purchase should reach Firebase/GA4 exactly once, with its real price and currency. Google Ads already reads the Firebase “Purchase” event, so once the event is right, purchases and revenue show up in Google Ads with no other setup.

1. What’s wrong today

- **Value missing.** Most “Purchase” events arrive without a value, so Google Ads records ₹1. Oct 27 – Nov 2: 48 purchases worth 371 in total; only about 2 carried a real value (~₹330). Values were mostly correct around Oct 5–20, then fell back to ₹1 in late October. Version 6.4.8.42 (released Oct 18) may have dropped the value; please check the event by app version in GA4.
- **Fired without a real purchase (earlier versions).** In June the event fired about 26 times a day; by October about 3–7 a day, while installs stayed at ~2,000 a day. Older versions likely fired on paywall views, failed attempts or restores.

2. Quick check first (may need no code)

Firebase automatically logs **`in_app_purchase`** for Google Play purchases, with value and currency. In GA4 > Reports > Engagement > Events, check whether `in_app_purchase` appears and carries a value. If it does and its daily count matches Play Console orders, we can import it into Google Ads instead of the custom event. Please confirm either way.

3. The “Purchase” event: when to fire

Keep the event name **`Purchase`** (exact spelling), so the existing Google Ads conversion keeps working.

- **Fire** in the Play Billing `PurchasesUpdatedListener` when `purchaseState == PURCHASED`, **after** the purchase is acknowledged (or consumed) successfully.
- **Fire once per purchase:** store the `purchaseToken` locally and never send the same token twice.
- **Do not fire** for: paywall or plan-screen views, `PENDING` purchases, cancellations or errors, restores (`queryPurchasesAsync` on app start), or subscription renewals (the app doesn’t see renewals; that’s fine for now).

4. Parameters to send

Parameter	Type	Value	Source (Play Billing Library)
`value`	number	Price paid, in currency units (e.g. 49000.0)	`priceAmountMicros / 1,000,000`: one-time `getOneTimePurchaseOfferDetails()`; subscription: the first **paid** pricing phase of the purchased offer
`currency`	string	ISO code, e.g. `IDR`	`priceCurrencyCode` of the same price
`transaction_id`	string	Order ID	`Purchase.getOrderId()`
`item_id`	string	Product ID	`Purchase.getProducts()`
`plan`	string	`weekly` / `monthly` / `6_month` / `yearly` / `lifetime` / `premium`	Your product mapping
`is_trial`	number	`1` if the purchase starts with a free trial,	Offer’s pricing phases

Parameter	Type	Value	Source (Play Billing Library)
		else `0`	

- **Currency is required.** Without it, IDR amounts can be read in the account currency (INR) and inflate revenue many times over.
- **Free trials:** send `value = 0` and `is_trial = 1`, and also log a separate **`trial_start`** event. The first paid charge happens later and isn't visible to the app.

5. Two small extra events (help bidding)

Event	When	Why
`paywall_view`	Each time the Pro / plans screen opens	Higher-volume signal closer to payment; lets us optimise for likely payers before purchases reach ~10 a day
`first_print_user`	Once per install, on the first successful print (store a flag)	Today "First_Print" counts every print, so we can't see the share of users who print. The SOP's activation goal (>20%) needs this

6. Optional: ad revenue into GA4 (not Google Ads)

Since ads are monetised through AppLovin MAX, its revenue callback can log ad revenue to Firebase, so GA4 shows ad revenue per user and per acquisition source:

- In `MaxAdRevenueListener.onAdRevenuePaid(ad)`, log Firebase's standard **`ad_impression`** event with `ad_platform = "appLovin"`, `ad_source` (network name), `ad_format`, `ad_unit_name`, `value = ad.getRevenue()` and `currency = "USD"`.
- Nothing is connected to Google Ads; this is for measurement only.

7. Testing

1. Use a Play Console license-tester account and buy each plan once (including one with a free trial, if offered).
2. Check Firebase DebugView: one `Purchase` per purchase, with `value`, `currency`, `transaction_id`, `item_id`, `plan`.
3. Restart the app after a purchase and confirm **no** second `Purchase` (restore must not fire it).
4. Open the plans screen and confirm `paywall_view`; do a first print on a fresh install and confirm `first_print_user` fires once.

8. Done when (7 days after release)

- At least 95% of `Purchase` events carry a value above 0 and a currency (trials aside).
- Daily `Purchase` count in GA4 is within about 10–15% of new orders in Play Console (all sources).
- No duplicate `transaction_id` values.

9. After release (Google Ads side, I'll handle)

Once the checks pass: set Android Purchase to count **once per install**, make it **Primary**, and start judging campaigns on cost per purchase and revenue per install. Until then, purchase value in Google Ads stays unreliable and I won't use value-based bidding.